



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

UNIVERSITÀ DEGLI STUDI DI TRIESTE

DIPARTIMENTO DI MATEMATICA, INFORMATICA E GEOSCIENZE

Corso di Laurea in Intelligenza Artificiale e Data Analytics

TESI DI LAUREA TRIENNALE

Conformal Inference as Regularization Term in Regression Models

Laureando:

Mattia Simonutti

MATRICOLA: SM3201292

Relatore:

Chiar.ma Prof.ssa Francesca Cairolì

ANNO ACCADEMICO 2023–2024

Il ringraziamento più grande v'è a mia mamma Cinzia, porto sicuro in un mare burrascoso, la sua presenza è di fondamentale importanza per me. Ringrazio Fulvio per la pazienza e i preziosissimi consigli che mi dona; Mio fratello Daniele per i momenti di leggerezza; La mia famiglia per l'amore e l'orgoglio che mi dimostra.

Ringrazio gli amici, Martina e Sara per la pazienza nell'ascoltarmi e nel darmi suggerimenti; Gli amici dell'università, Gabri, Davide, Kj, Sara, Lenox e Rita nello specifico, per avere reso questo percorso più leggero ed allegro; Filippo e Canto amici fidati e infine ma non per importanza il mio socio Manuel.

Voglio dedicare la raggiunta di questo traguardo a 3 persone importantissime: Mio papà Mauro, il destino ci ha separati troppo presto ma sento la tua presenza costantemente; Mio nonno Guido che aveva pronosticato questo giorno con ben 20 anni di anticipo; e infine mio zio Renzo che mi ha sempre spronato a portare a termine questo percorso. Vi porto sempre nel cuore.

Non potevo non menzionare la mia compagna di studio e stesura della tesi, la mia gatta Patty. Il sistema di distribuzione dei gatti mi ha benedetto e io ne sono grato.



Abstract

Una delle maggiori sfide dei giorni nostri è quella di ottenere previsioni accurate e affidabili in diversi ambiti di studio e lavoro. Per garantire un'alta qualità è fondamentale associare alle previsioni un grado di incertezza, che indichi quanto siano affidabili tali risultati. In questo lavoro viene presentata l'applicazione della teoria di Conformal Prediction (CP) a problemi di regressione. La Conformal Prediction (CP) è una tecnica statistica che consente di ottenere, nel caso della regressione, intervalli di previsione affidabili, con garanzie formali di copertura. A differenza di molti approcci tradizionali, la CP non richiede ipotesi stringenti sulla distribuzione dei dati, rendendola particolarmente versatile per applicazioni in cui tali ipotesi potrebbero non essere soddisfatte. L'obiettivo di questa tesi è quello di rendere gli intervalli di previsione il più efficienti possibili, evitando di sovrastimare l'incertezza, pur preservando le garanzie statistiche di CP.

Contents

Abstract	I
Contents	II
Introduction	III
0.0.1 Why is regression analysis so important?	III
0.0.2 Objective of the study	IV
1 Theoretical fundamentals	1
1.1 Supervised Learning	1
1.1.1 Objective in supervised learning	1
1.1.2 Artificial Neural Networks	3
1.2 Linear Regression	6
1.2.1 Simple Linear Regression	7
1.2.2 Multiple Linear Regression	9
1.2.3 Polynomial regression	10
1.2.4 Evaluation of the model	12
1.3 Conformal Inference	14
1.3.1 Foundations of conformal inference	14
1.3.2 Inductive Conformal Prediction	16
2 Methodology: Conformal Prediction for Regularization	18
2.1 Regularization for Classification Problems	18
2.1.1 Conformal Training: ConfTr	18
2.2 Regularization for Regression Problems	20
2.2.1 Results	22
2.3 Conclusions	24

Introduction

Regression analysis is a widely used set of supervised machine learning tools designed to estimate relationships between a dependent variable, often referred to as the outcome, and one or more independent variables, commonly known as features. These are crucial tools for a variety of sectors where accuracy and consistency in predictions are essential, including engineering design, financial forecasting and medical diagnosis just to name a few.

In many domains, outcomes and decision-making can be strongly influenced by the precision of forecasts and that's why the use of Artificial Neural Networks (ANNs) has increased significantly. Neural networks' flexibility and capacity to represent intricate linear and non-linear relationships in data allow them to achieve high accuracy and precision. Even with their achievements, neural networks frequently encounter problems with coverage guarantees and uncertainty quantification. This makes it more difficult to safely and successfully deploy results obtained in practical settings, where the repercussions of making erroneous predictions might be severe such as in the medical field. Therefore, addressing the issues of uncertainty and reliability is essential.

Conformal Inference (or conformal prediction) presents a viable remedy in this situation as it can improve neural network prediction reliability by offering statistically solid techniques to rigorously quantify the prediction uncertainty, thing that neural networks struggle to do. Conformal Inference (or conformal prediction) presents a viable remedy in this situation as it can improve neural network prediction reliability by offering statistically solid techniques to rigorously quantify the prediction uncertainty, thing that neural networks struggle to do.

0.0.1 Why is regression analysis so important?

Regression analysis comprehends a wide range of tools, such as linear regression, polynomial regression, Poisson regression, etc...Regression analysis has a great importance in several fields. Firstly, it has pure statistical uses:

- It can be used to predict future outcomes or identify missing data;
- Can be used to summarize or describe the observed data.
- It can help to measure the strength of the relationship between two sets of variables by identifying the R-square value.

INTRODUCTION

In second place regression analysis has a wide range of applications such has:

- Economics: Modeling the relationship between GDP and other economic indicators;
- Finance: Predicting stock prices based on historical data;
- Medicine: Relating dosage levels of a drug to its effectiveness, predict the possibility of developing a certain disease due some information on the patience medical history.
- Forecasting: determine which of these variables are likely to have the biggest impact based on previous events and help make more accurate forecasts and predictions.

It is of great interest therefore to research, deepen and improve modern regression techniques

0.0.2 Objective of the study

In this paper, we attempt to increase the reliability of the predictions obtained by reducing the uncertainty on the results provided by a neural network for regression problems. This is due to the tendency of neural networks to maximize the accuracy of predictions without taking into account the level of uncertainty of the predictions made.

The main idea is to use conformal prediction as a regularization term during the training phase of the neural network. This is aimed to minimize errors on the predictions (maximize the accuracy of our network) and at the same time provide targeted confidence intervals to indicate how reliable and accurate the predictions obtained are.

Chapter 1

Theoretical fundamentals

1.1 Supervised Learning

Machine learning is an application of artificial intelligence (AI) that provides machines the ability to learn from data without being explicitly programmed towards it. Machine learning have seen its popularity heavily increased in recent years mostly to three extremely important factors:

- Data abundance, in recent years the quantity of data produced by humans and their applications is growing exponentially giving machine learning algorithms a vastly amount of data to use in training phases.
- Algorithmic developments, researchers have been studying new methods to improve algorithms and their performances.
- Better computers, computer manufacturing has increased their effectiveness in producing more powerful and reliable computing instruments.

Machine learning is being used in many many applications such has, medical diagnosis, stock market trading, self driving cars and a lot of others important fields. Machine learning can be divided in three big branches, Supervised Learning, Unsupervised Learning and Reinforcement Learning.

In this work we will focus on linear regression and consequently on supervised learning. Supervised Learning implies that the data, correctly labelled, are being given at start the study by an external teacher that supervise the training of the model.

Supervised Learning focuses then on two great fields of study, Regression Analysis and Classification tasks

1.1.1 Objective in supervised learning

The training data in supervised learning setup comes in pairs of inputs (x, y) , where $x \in \mathbb{R}^d$ is the input instance and $y \in \mathbb{R}$ its target, a label in classification tasks or a number in regression problems. The complete training dataset is

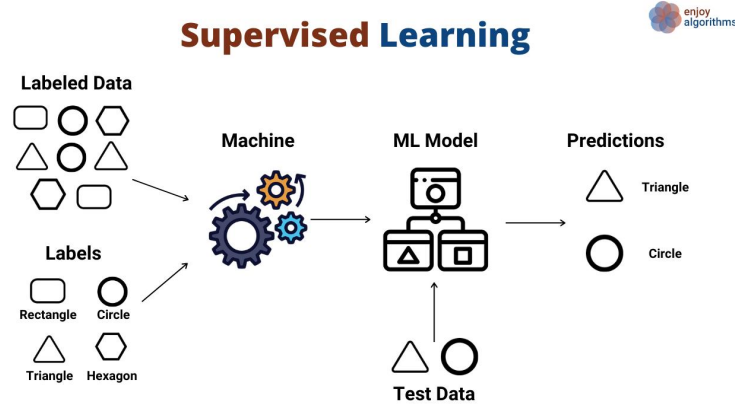


Figure 1.1.1: Supervised learning training scheme

composed as:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \subseteq \mathbb{R}^d \times \mathcal{C}$$

where:

- $\mathbb{R}^d$ is the d-dimensional feature space
- $\mathcal{C}$ is the target space

The pairs (x_i, y_i) are drawn from an unknown distribution $\mathcal{P}(X, Y)$ and, through the training of a supervised machine learning model we would like to learn a function f , such that for a new observation $\mathbf{x}$ and its unseen target y we have $f(\mathbf{x}) \approx y$.

How do we obtain the function f ?

In order to get the model to learn something from the data we must guide it through the learning phase using a problem-specific loss.

For linear regression we formulate again the model and the loss function but with machine learning notation:

$$y = \omega^T x + b$$

$$\mathcal{L}(\omega, b) = \frac{1}{N} \sum_{i=1}^N (y_i - (\omega^T x + b))^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

We want the ω coefficients to be such that the loss is minimum and we do it by computing the loss gradient, evaluating it with the current coefficients and then using the value obtained to update the weights (another way to call ω).

This procedure is known as Gradient-Descent:

1. Fix randomly $\omega_{t=0}$

CHAPTER 1. THEORETICAL FUNDAMENTALS

2. WHILE (condition):

- (a) Compute gradient at ω_t : $\nabla \mathcal{L}(\omega_t)$;
- (b) Update weights $\omega_{t+1} = \omega_t - \gamma \frac{\delta \mathcal{L}(\omega, b)}{\delta \omega}$

It is important to mention three important things about this approach:

- **Initialization:** Different initial values of the weights can lead to very different solutions involving the ones which are not good enough. Several runs can help prevent this inconvenient.
- **Learning Rate γ :** It is important to choose carefully the learning rate, a too high value would make the loss value fluctuate without reaching the minimum while a too low value would make the model training too slow.
- **Termination condition:** when to stop the G-D algorithm. There are several indicators to look at, number of steps, minimum acceptable loss value reached ect...

The model evaluation is based on the R^2 and the adjusted- R^2 values discussed in section 2.1.3.

1.1.2 Artificial Neural Networks

Artificial neural networks (ANN), or simply neural networks is a machine learning model created through researchers' attempt to recreate the behavior of neurons and the human brain, with the intent of emulating its ability to solve complex problems and process information at high speed and accuracy. An important theorem (López, López and Crossa [1]) justifies the great importance that neural networks have:

Theorem 1.1.1 (*Universal Approximation Theorem [1]*) *Let $g()$ be a bounded, and monotone-increasing continuous function. Let I_{m_0} denote the m_0 -dimensional unit hypercube $[0, 1]^{m_0}$. Given any function $f \in \mathcal{C}(\mathcal{I}_{\mathbb{R}})$ and $\epsilon > 0$, there is an integer m_1 and sets of real constants a_i, b_i and w_{ij} , where $i = 1, \dots, m_1$ and $j = 1, \dots, m_0$ such that we may define:*

$$F(x_1, \dots, x_{m_0}) = \sum_{i=1}^{m_1} a_i g\left(\sum_{j=1}^{m_0} w_{ij} x_j + b_i\right)$$

as an approximate realization of function $f()$; that is,

$$|F(x_1, \dots, x_{m_0}) - f(x_1, \dots, x_{m_0})| < \epsilon$$

For all $x_1, \dots, x_{m_0}$ that lie in the input space.

m_0 represents the input nodes of a multilayer perceptron with a single hidden layer. m_1 is the number of neurons in the single hidden layer, $x_1, \dots, x_{m_0}$ are the inputs, w_{ij} denotes the weight of neuron i in input j , b_i denotes the bias corresponding to neuron i , and a_i is the weight of the output layer in neuron i .

CHAPTER 1. THEORETICAL FUNDAMENTALS

The theorem states that a neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function over a compact subset of the inputs.

Structure of a Neural Network

The fundamental element of neural networks are the artificial neurons (figure: 1.1.2), functions that perform a weighted summation over the inputs they receive applying to the result of it an activation function that keeps the output of the neuron within a certain range. Activation functions allow nonlinearities to be introduced into the network's modeling capabilities.

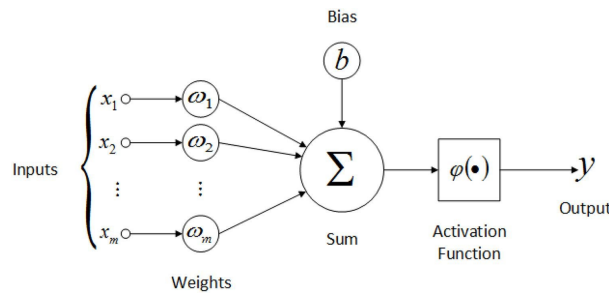


Figure 1.1.2: Artificial Neuron scheme

Other important components in an artificial neural network are:

- Input layer: set of neurons that receive the initial data fed to the neural net. Each neuron in this layer corresponds to a feature in the input data.
- Hidden layer: those layers, composed by neurons, are placed between the input set and the outcome set and perform non-linear transformations of the input data through weighted connections. The depth (number of hidden layers) and width (number of neurons per layer) can vary depending on the complexity of the problem.
- Output layer: The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of output classes in classification tasks or to a single neuron in regression tasks.

The structure of a neural network is composed by inter-connected (synapses) artificial neurons that communicate with each other sharing informations.

There are two kinds of neural networks:

- shallow neural network, in this architecture we find one input layer, one hidden layer and one output layer;
- deep neural network, in this case the only difference is in the number of hidden layers > 1

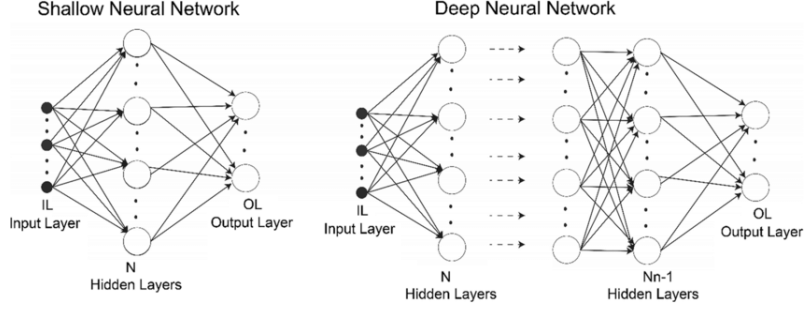


Figure 1.1.3: Shallow Vs Deep

Training of a Neural Network

The objective of the net's training is to obtain the right coefficients ω as to minimize the function loss value. The training of a neural network follows four major steps:

1. **Forward Pass:** The input is passed through the entire neural network until an output is obtained. During this passage, all the outputs of each layer (activations) are stored;
2. **Evaluation of the error:** The differences between the neural net output and the true outcome are obtained and used to evaluate how far the predictions are from the desired values;
3. **Backward pass:** The gradient of the loss function is calculated with respect to the network output and, through the chain rule, then it is propagated backwards through the network, layer by layer. The gradient is calculated with respect to each weight and bias, based on the gradients received from the next layer and the activations stored during the forward pass;
4. **Weight's update:** The calculated gradients are used to update the weights and biases of each layer through an optimization algorithm, such as gradient descent. The goal of the update is to reduce the loss.

Several optimizers exist, allowing the model to look for the best coefficients ω that minimize the loss value function:

- Batch GD: computes the gradient of the cost function w.r.t to the parameters ω for the entire training dataset:

$$\omega = \omega - \gamma \nabla \mathcal{L}(\omega, x, y)$$

Since for a single weight update we need to evaluate the gradient on the complete training set this technique can be very slow;

- Stochastic GD: performs a parameter update for each training example x_i and label y_i .

$$\omega = \omega - \gamma \nabla \mathcal{L}(\omega, x_i, y_i)$$

CHAPTER 1. THEORETICAL FUNDAMENTALS

It is much faster but its results have great variance causing the loss function to fluctuate heavily;

- Mini-Batch GD: it combines both the characteristics of the previous methods:

$$\omega = \omega - \gamma \nabla \mathcal{L}(\omega, x_{(i:i+n)}, y_{(i:i+n)})$$

It reduces the variance of the parameters update while improving the computation of the gradients.

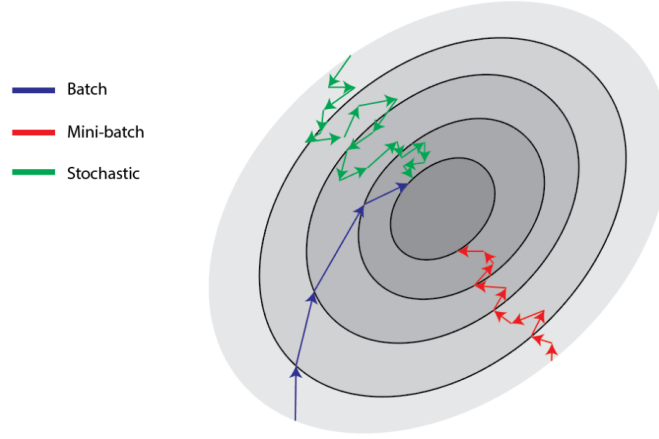


Figure 1.1.4: Loss values with different optimizers

1.2 Linear Regression

Despite being used for over a hundred years Linear Regression still holds as one of the preferred models to establish a relationship between different variables as it combines ease of use and interpretability with great prediction abilities (it is important that the linearity condition between the independent and dependent variables holds).

The linear regression model assumes that, given a data set $D = \{(x_i, y_i)\}_{i=1}^n \subset \mathbb{R}^d \times \mathbb{R}$, where $x \in \mathbb{R}^d$ is a feature vector and $y \in \mathbb{R}$ is the corresponding outcome or label, the relationship between the feature vector and the outcome is linear. Thus the model assumes the following mathematical formulation:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id} + \epsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \epsilon_i \quad i = 1, \dots, n$$

where $\mathbf{x}_i^T$ indicates the transpose of the i^{th} feature vector.

The random variable ϵ is the error term in the model. In this context, error does not mean mistake but is a statistical term representing random fluctuations, measurement errors, or the effect of factors outside of our control.

We can write the above equation in matrix form:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 & x_{11} & \dots & x_{1d} \\ 1 & x_{21} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{nd} \end{bmatrix},$$

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_d \end{bmatrix}, \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_0 \\ \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}.$$

1.2.1 Simple Linear Regression

The primary objective of the linear regression model is to predict future values of a variable of interest given different values of one or more independent variables. In order to guide our regression model through the training phase, searching for the best coefficients, we use a loss function called Mean Squared Error.

In the simplest cases, simple linear regression, we have just one feature $x \in \mathbb{R}^n$ and one outcome $y \in \mathbb{R}$. The model and the loss function have the following shape :

$$y = \beta_0 + \beta_1 x + \epsilon$$

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

$$\mathcal{L}(y, \hat{y}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The term $\hat{y}$ is the prediction obtained and n is the number of outputs. In this case it's possible to estimate the correct coefficients just by doing some simple derivatives:

$$\begin{cases} \frac{\delta \mathcal{L}(y, \hat{y})}{\delta \hat{\beta}_0} = \sum_{i=1}^n -2(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0 \\ \frac{\delta \mathcal{L}(y, \hat{y})}{\delta \hat{\beta}_1} = \sum_{i=1}^n -2x_i(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0 \end{cases}$$

that is equivalent to:

$$\begin{cases} \sum_{i=1}^n y_i - n\hat{\beta}_0 - \sum_{i=1}^n \hat{\beta}_1 x_i = 0 \\ \sum_{i=1}^n x_i y_i - \hat{\beta}_0 \sum_{i=1}^n x_i - \sum_{i=1}^n \hat{\beta}_1 x_i^2 = 0 \end{cases}$$

By solving the system of least squares normal equations the coefficients obtained are:

CHAPTER 1. THEORETICAL FUNDAMENTALS

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad \hat{\beta}_1 = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \hat{\rho} \sqrt{\frac{Var(Y)}{Var(X)}}$$

where $\hat{\beta}_0$ is the estimated coefficient for the intercept and $\hat{\beta}_1$ is the estimated slope coefficient of the regression line. Pearson's correlation coefficient, $\hat{\rho}$, measures the linear correlation between two variables.

$$\hat{\rho} = \frac{cov(X, Y)}{\sqrt{V(X)V(Y)}} \quad -1 \leq \hat{\rho} \leq 1$$

The closer $\hat{\rho}$ is to ± 1 the more stronger the dependence between the two variables is (positive if closer to 1 or negative if closer to -1).

The estimated regression line is therefore:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

Note that for each pair of observations $(y_i, x_i)_{i=1}^n$ the following relationship is satisfied:

$$y_i = \hat{\beta}_0 + \hat{\beta}_1 x_i + e_i \quad i = 1, \dots, n$$

where $e_i = y_i - \hat{y}_i$ is called the residual. The residual measures the discrepancy between the true observed value and the value obtained through the regression model. They are used to evaluate the adequacy of the fitted model.

Interpretation of the coefficients

The intercept term, β_0 in a regression table tells us the average expected value for the response variable when all of the predictor variables are equal to zero. It's important to note that the regression coefficient for the intercept is only meaningful if it's reasonable that all of the predictor variables in the model can actually be equal to zero. It may be useful sometime to center the data around 0 by subtracting to every single value in the data set the mean of all the data.

On the other hand the slope coefficient, β_1 suggest the relationship between the response variable and the independent variable. For example if the regression model estimates a value of 0.8 for the slope, the relation between the outcome and the regressor is positive, meaning that an increase of 1 unit on the value of the regressor corresponds to an increase of 0.8 in the value of the outcome we are interested in.

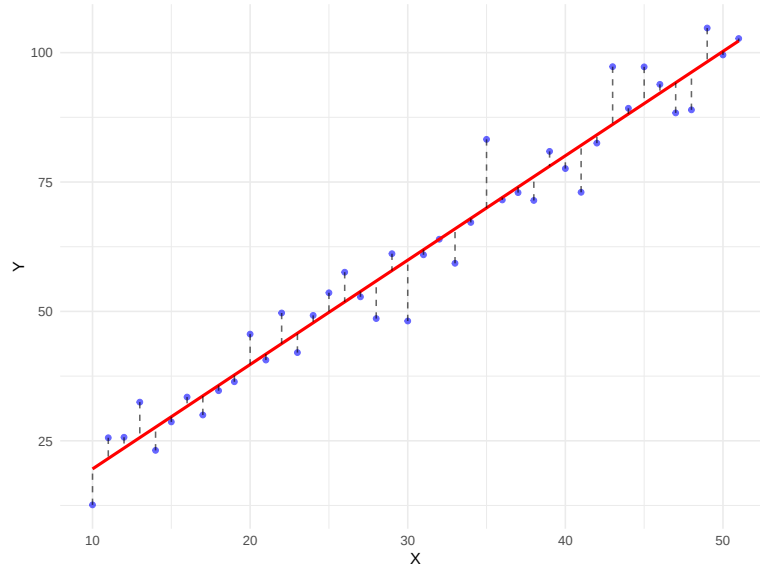


Figure 1.2.5: Example of simple linear regression and residuals

1.2.2 Multiple Linear Regression

In general, the response variable y is often influenced by more than one predictor variable as in Figure 1.2.5, this case is referred to as Multiple Linear Regression.

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d + \epsilon = \beta_0 + \sum_{i=1}^d x_i \beta_i + \epsilon$$

The loss function doesn't change.

$$\mathcal{L}(y, \hat{y}) = \sum_{i=1}^n (y_i - x_i \hat{\beta}_i)^2 = \min_{\beta} \|\epsilon\|_2^2$$

we want to find the right coefficients β in order to minimize the squared differences between the predictions obtained from our model and the true values given. The coefficients can be obtained through the same steps shown in simple linear regression but it would require to calculate several derivatives.

The interpretation of the coefficients remains the same as the simple linear regression's, the value of a coefficient of a certain feature suggests the influence of that variable on the response variable

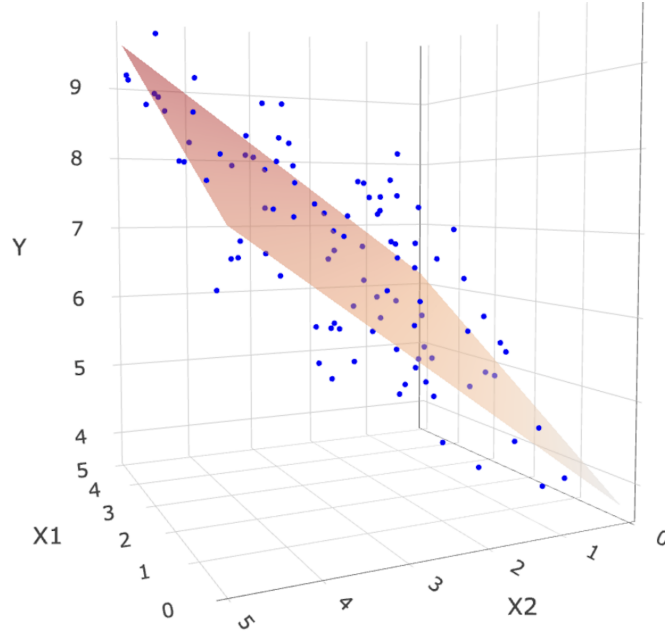


Figure 1.2.6: Example of multiple linear regression

1.2.3 Polynomial regression

A more typical framework is met when the relationship between the response variable and the independent variables is not linear (degree $d = 1$) but is modeled as a polynomial of degree d .

Types of Polynomials

$$\text{Linear} \rightarrow y = \beta_0 + \beta_1 x$$

$$\text{Quadratic} \rightarrow y = \beta_0 + \beta_1 x + \beta_2 x^2$$

$$\text{Cubic} \rightarrow y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

The goal in this framework is the same as the other regression models in regression analysis, find a suitable relationship between the response variable and the independent variables, minimizing a certain loss function and use the final model obtained to obtain future predictions. More in general we can write:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d + \epsilon$$

The above equation can be expressed in matrix form too:

$$\mathbf{Y} = \beta \mathbf{X} + \epsilon$$

where,

$$\mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 & x_{11} & \dots & x_{1d}^d \\ 1 & x_{21} & \dots & x_{2d}^d \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{nd}^d \end{bmatrix},$$

CHAPTER 1. THEORETICAL FUNDAMENTALS

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_d \end{bmatrix}, \quad \epsilon = \begin{bmatrix} \epsilon_0 \\ \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}.$$

The loss function used is the same as the one shown in the previous sub chapters, the MSE.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

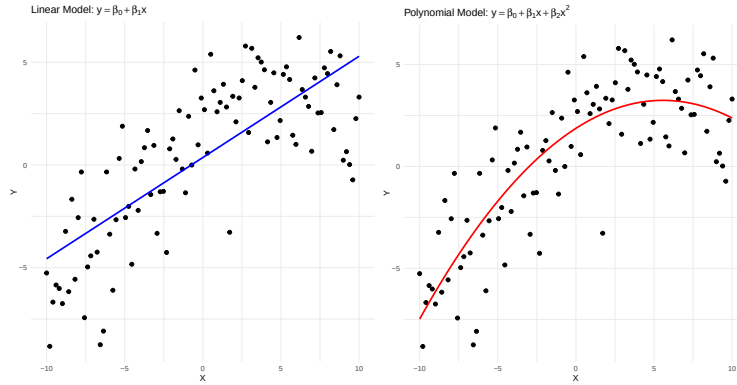


Figure 1.2.7: Linear vs Polynomial

Polynomial regression has several advantages:

- Flexibility: It can model a wide range of relationships, from linear to high dimension non-linear;
- Improved Fit: It can provide a better fit to data that exhibits curvature or non-linear patterns, coming in clutch to simple and multiple linear regression;
- Feature Engineering: it can help transform original features into higher-order terms, which can be useful in capturing complex underlying patterns.

But it comes with some disadvantages too:

- Overfitting: Higher-degree polynomials can lead to overfitting, where the model perfectly learns the noise in the training data without learning a pattern in the data, causing the predictions in the test phase to be heavily wrong;
- Interpretability: As the degree of the polynomial increases, the model becomes harder to interpret and explain;
- Data Availability: Higher-degree polynomials require more data points to accurately estimate the coefficients, making them less suitable for small datasets.

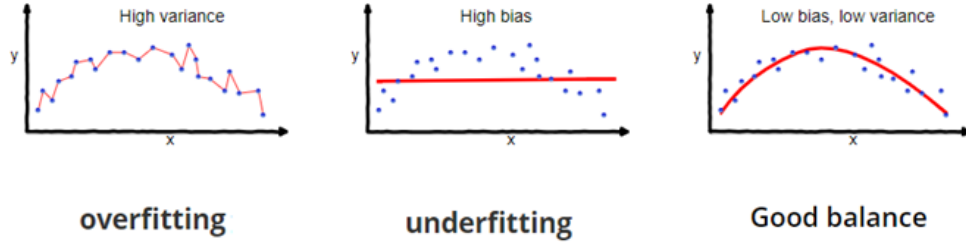


Figure 1.2.8: Underfitting, overfitting and just good model

It's important to balance the bias and the variance of the model in order to achieve the good balance showed in 1.2.8.

1.2.4 Evaluation of the model

Once the final model is obtained we are interested in evaluating how good it is. We do this check by analyzing both the residuals of the model and the R^2 value. In order to obtain the R^2 value we need two measures, the squared sum of residuals (SSR) and the total sum of squares (TSS) :

$$SSR = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad TSS = \sum_{i=1}^n (y_i - \bar{y})^2$$

So the R^2 is obtained:

$$R^2 = 1 - \frac{SSR}{TSS}$$

The value obtained is the fraction of the variance that the fitted regression model was able to explain. The more regressors we insert into the model the more variance explained we expect to achieve, we want then a measure to balance the fraction of variance explained and the number of variables inserted into the model. The value to measure this balance is the adjusted- R^2 .

$$R_{adj}^2 = 1 - \frac{\frac{SSR}{df_{model}}}{\frac{TSS}{df_{mean-model}}}$$

where df_{model} is the degrees of freedom of the regression model and $df_{mean-model}$ is the degrees of freedom of mean model. If the model has p regression variables and a data set of size N the two values would be $df_{model} = (N - 1 - p)$ and $df_{mean-model} = (N - 1)$. R_{adj}^2 has some problems, notably:

- It treats the effect of all regression variables equally. In reality, some variables are more influential than others in their ability to make the model fit (or over-fit) the training data;

CHAPTER 1. THEORETICAL FUNDAMENTALS

- The formula for R_{adj}^2 yields negative values when R^2 falls below $\frac{p}{N-1}$ thereby limiting the use of R_{adj}^2 to only values of R^2 that are above $\frac{p}{N-1}$.

It's a good habit to check both the values to have a reference about the goodness of the regression model.

Residual's Assumptions

In addition to the measures of variance explained it's important that the residuals meet certain requirements:

- **Non-correlation of residuals:** Residuals must be independent of each other, if this condition is not satisfied there may be some problems with hypothesis testing and R^2 value may not be correctly evaluated;
- **Homoscedasticity of residuals:** Residuals must have the same variability along the entire range of values predicted by the model. If this requirement is not met the model's coefficient may be biased and the results obtained through statistical tests may be not as strong;
- **Gaussian distribution of residuals:** Residuals must follow the gaussian distribution $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, in order to guarantee the validity of the results obtained from the statistical tests;

We can check if those requirements are satisfied looking at the residuals plots shown in figure 1.2.9. We use the first plot to verify the linearity of the variables of the model through checking for the linearity of residuals, the second one to check for residual's gaussianity and the third one to check for heteroscedasticity of the residuals.

If the residuals assumptions holds and the R^2 or the adjusted- R^2 measurements are significant we can assume that the model is good and so are the predictions obtained through it.

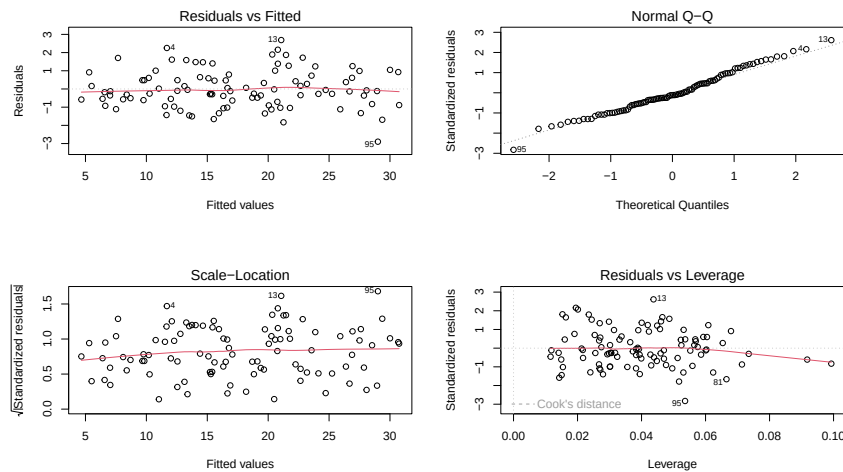


Figure 1.2.9: Residuals plot for assumptions check

1.3 Conformal Inference

Nowadays machine learning models are widely used for modelling and prediction aims, even in high-risk settings such as medical diagnosis, prediction of stock markets values and engineering, all subjects which require the predicted outcomes to come with an uncertainty estimation in order to quantify how reliable the prediction is, avoiding potentially catastrophic results.

Conformal Inference (also known as conformal prediction) offers a robust framework to address this needs by providing prediction intervals or sets with guaranteed coverage probabilities for such duties. It is a statistical method designed to produce valid prediction intervals or sets that have a user-specified probability of containing the true outcome.

1.3.1 Foundations of conformal inference

Conformal predictors

We start with a set of data

$$\{x_i, y_i\}_{i=1}^n$$

where $x_i \in \mathcal{X}$ (feature space) and $y_i \in \mathcal{Y}$ (label space) $\forall i, i = 1, \dots, n$.

We make two important assumptions, the randomness assumption implies that every value $z_i = (x_i, y_i)$ is drawn with equal probability and comes from the same distribution $\forall i$, i.e. the variables are said to be independent identically distributed (i.i.d); the second assumption required is the exchangeability, the sequence $(z_1, z_2, \dots)$ is drawn from a probability distribution that is exchangeable: for any permutation π of the set $\{1, \dots, n\}$, the joint probability distribution of the permuted sequence $(z_{\pi(1)}, \dots, z_{\pi(n)})$ is the same as the distribution of the original sequence.

Suppose we are given a sequence $x_1, y_1, \dots, x_{n-1}, y_{n-1}, x_n$ and we want to predict y_n , we need a function

$$\mathcal{C} : \mathbf{X} \rightarrow \mathbf{Y}$$

called simple predictor. This function gives $\mathcal{C}(x_1, y_1, \dots, x_{n-1}, y_{n-1}, x_n) \in \mathcal{Y}$ as its prediction for the new label y_n .

We want to give subsets of $\mathbf{Y}$ large enough that we can be confident that y_n will fall in them, while also giving smaller sub-sets in which we are less confident. An algorithm that predicts in this sense requires an additional input $\alpha \in (0, 1)$, which we call the significance level; the complementary value $1 - \alpha$ is called the confidence level.

So we want an algorithm $\mathcal{A}$ such that

$$\mathcal{A}(x_1, y_1, \dots, x_{n-1}, y_{n-1}, x_n, \epsilon) \in \mathcal{Y}$$

It is also important to note that the algorithm must satisfy:

$$\mathcal{A}(x_1, y_1, \dots, x_{n-1}, y_{n-1}, x, \alpha_1) \subseteq \mathcal{A}(x_1, y_1, \dots, x_{n-1}, y_{n-1}, x, \alpha_2) \quad (2.3.1.1)$$

CHAPTER 1. THEORETICAL FUNDAMENTALS

whenever $\alpha_1 \geq \alpha_2$; The smaller α the higher the confidence prediction is.

We call a function

$$\mathcal{A} : \mathbf{X} \times (0, 1) \rightarrow 2^{\mathbf{Y}},$$

where $2^{\mathbf{Y}}$ is the set of all subset s of $\mathbf{Y}$, that satisfies (2.3.1.1) a confidence predictor.

Nonconformity scores

Nonconformity scores are maps:

$$S : \mathcal{X}^n \times \mathcal{Y} \rightarrow \mathbb{R}$$

used to measure how different an example $z = (x, y)$ is from a bag $B \in \mathcal{X}^n \times \mathcal{Y}$ of old examples.

In regression problems, a suitable nonconformity score that can be used is the absolute value of the difference between y and the predicted value $\hat{y}$ calculated from x and the old examples: $s(x, y) = |f(x) - y|$

Once the nonconformity scores are computed, we would like to know how unusual the value predicted by the model is. One way of doing it is by calculating the fraction:

$$\frac{|\{j = 1, \dots, n : \alpha_j \geq \alpha_i\}|}{n}$$

This value, that lies between $\frac{1}{n}$ and 1, is called the $p - value$ for z_i . It's the proportion of examples in the bag as nonconforming as z_i . If this value is small ($p \approx \frac{1}{n}$), z_i is heavily non-conforming and it is likely not to be included in the prediction set, on the other hand if the $p - value$ is high enough, z_i is more likely to be added to the prediction set. Hence we define the prediction set that satisfies the coverage requirements as:

$$\gamma^\alpha(z_1, \dots, z_n) := \{z \in Z : p_z \geq \alpha\}$$

Evaluating the prediction set obtained

Validity and efficacy represent the measurements used in conformal inference to evaluate the performances of the obtained confidence predictors.

For each significance level α , we want to have confidence $1 - \alpha$ in our prediction about y_n . In other words, this means that the difference between the probability of a prediction error and the probability of the event should be α . Moreover, since we are making a whole sequence of predictions, $y_1, y_2, \dots$ we would like these error events to be independent. If these conditions hold, we can affirm that our confidence predictor $\mathcal{A}$ is "exactly valid".

In order to be more clear we give a formal definition: Given a data sequence,

$$\omega = (x_1, y_1, x_2, y_2, \dots)$$

CHAPTER 1. THEORETICAL FUNDAMENTALS

with a significance level α , we define the error that the conformal predictor makes on the n^{th} trial as:

$$err(\mathcal{A}, \omega, \alpha, n) := \begin{cases} 1 & \text{if } y_n \notin \mathcal{A}(x_1, y_1, x_2, y_2, \dots, x_{n-1}, y_{n-1}, x_n, \alpha) \\ 0 & \text{otherwise,} \end{cases}$$

and the number of total errors during the first n trials is:

$$Err(\mathcal{A}, \omega, \alpha, n) := \sum_{i=1}^n err(\mathcal{A}, \omega, \alpha, i)$$

An important result holds:

Proposition 1.3.1 *Under the exchangeability assumption, the probability of error, $z_{n+1} \notin \gamma^\alpha(z_1, \dots, z_n)$, will not exceed α , for any α and any conformal predictor γ .*

Giving a bound to the error that can be encountered using conformal predictors.

Efficacy is another important, although not as important as validity, measure of the goodness of the conformal predictor γ . We take into account two of the problems of machine learning, classification and regression:

- Classification: The most used indicator for efficiency is the size of the prediction set, the more elements are present in the prediction set the less efficient the conformal predictor is;
- Regression: The prediction set in these problems is an interval of values and a natural measure of the efficiency of such a prediction is simply the length of the interval. The smaller the interval, the greater the ability of the predictor is.

Instructions for conformal prediction

CP can be used on top of any prediction or classification algorithm and moreover on data drawn from any distribution without constraints. The steps to perform it are:

- Make predictions or classifications using a pre-trained model;
- Calculate the conformity scores $s(x, y) \in \mathbb{R}$ of predictions;
- Compute the desired quantile $\hat{q}$ of the conformity scores;
- Use the quantile obtained to create the prediction set for new examples:

$$\mathcal{C}(x_{test}) = \{y : s(x_{test}, y) \leq \hat{q}\}$$

1.3.2 Inductive Conformal Prediction

Inductive Conformal Prediction (ICP) is a set of distribution-free and model agnostic algorithms which is aimed to provide predictions accompanied by a user-defined confidence level with coverage guarantee. Depending on the problem

CHAPTER 1. THEORETICAL FUNDAMENTALS

encountered, this set of algorithms return an interval (regression analysis) or a set of classes (classification purposes). An element of great importance in IPC is the calibration set I_{cal} ; we use this set in order to obtain the correct quantile $\hat{q}$ that will be used to construct the final prediction set/interval that guarantees the confidence and the coverage required by the user.

Classification Algorithm:

- Given the training data, split them into two smaller sets, a training set I_{train} and a calibration set I_{cal} ;
- Train the neural network with the training data I_{train} ;
- Compute the predictions for the calibration set I_{cal} using the trained neural network and then obtain the nonconformity scores s_i of every couple. The non-conformity scores used are $s(x, y_{true}) = 1 - f(x)_{y_{true}}$;
- Given the non-conformity scores of the calibration set take the quantile $\hat{q}$ that guarantees the desired coverage. The parameter α given by the user indicates the required coverage;
- Feed the test examples to the neural network, using the predictions obtained calculate the non-conformity scores;
- Form the final prediction set $\mathcal{C}(X_{test}) = \{y : \hat{f}(X_{test_y}) \geq 1 - \hat{q}\}$

The regression algorithm follows the same steps, the thing changing is the way we compute the non-conformity scores as $s(x, y_{true}) = |f(x) - y|$ and the way we construct the confidence intervals such as $[f(x_i) - \hat{q}, f(x_i) + \hat{q}]$.

Chapter 2

Methodology: Conformal Prediction for Regularization

2.1 Regularization for Classification Problems

In this work, we try to extend the work done by (Stutz et al.,2022) that aims to differentiate through conformal prediction (CP) during training with the goal of training the model with the conformal wrapper end-to-end. Their work aims to guarantee the safe deployment of the results obtained through the model for classification problems, simulating conformalization on mini-batches during the model's training. With this idea, the algorithm they proposed, ConfTr (conformal training) outputs prediction sets with reduced average confidence set size (inefficiency). Moreover, ConfTr algorithm allows to shape the confidence sets predicted at test time, meaning that the ConfTr algorithm not only trains the model to make accurate predictions but also allows for fine-tuning the model's output to balance confidence and precision in a way that is most appropriate for the specific application and the data at hand.



Figure 2.1.1: Examples of prediction sets, from [2]

2.1.1 Conformal Training: ConfTr

Conformal Training's main objective is to guarantee safe deployment of results obtained for classification problems, including modern conformal prediction meth-

CHAPTER 2. METHODOLOGY: CONFORMAL PREDICTION FOR REGULARIZATION

ods during the training phase of a machine learning model, minimizing the inefficiency and accompanying results with their uncertainty.

In order to include conformal prediction in the training phase, the conformal predictors used must be differentiable (otherwise backpropagation and the update of the neural net weights wouldn't be possible). It's important then to define conformal predictors $\mathcal{C}_\omega(X; \tau)$: which depend on the model parameters ω through the predictions π_ω and where the threshold τ is determined during calibration on a held-out calibration set (X_i, Y_i) , $i \in \text{Ical}$ to obtain coverage.

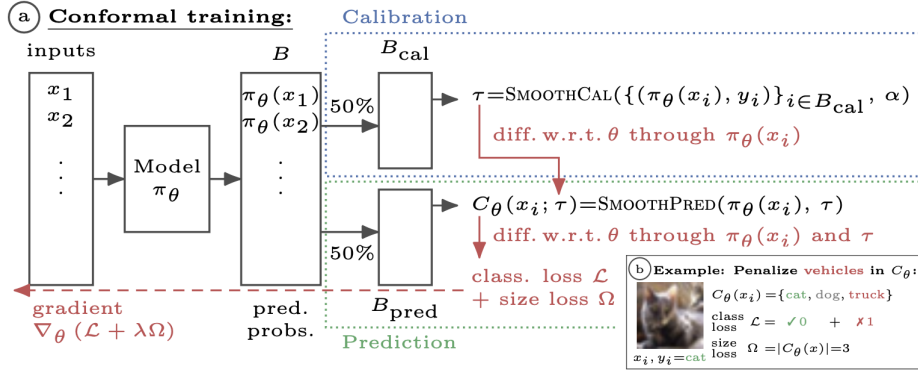


Figure 2.1.2: Conformal training, thanks to [3]

The prediction obtained from the model are divided into two batches B_{cal} , the calibration set used to obtain τ , the threshold used for constructing the intermediate and final prediction set, and B_{pred} the intermediate prediction set, composed by the predicted elements which have a p-value higher then the threshold τ .

ConfTr uses a conformal predictor called threshold conformal predictor (THR) (Sadine et al., 2019) that constructs the prediction sets by thresholding probabilities:

$$\mathcal{C}_\omega(X, \tau) := \{k : \pi_{\omega,k}(x) =: E_\omega(x, k) \leq \tau\}$$

where E_ω is the conformity score of example x classified in class k . τ is computed during the calibration step as the $\alpha(1 + \frac{1}{|B_{cal}|})$ quantile of the conformity scores $E(x_i, y_i) = \pi_{\omega, y_i}(x_i)$

Differentiating through CP requires the intermediate steps, Prediction and Calibration, to be differentiable:

- **Prediction:** it involves thresholding conformity scores. This operation can be smoothed (becoming differentiable) by using a smoothing function, in this work the function used is the sigmoid function and a parameter called temperature T . Since for THR holds $E_\omega(X, k) = \pi_{\omega,k}(x)$ this step is already differentiable;
- **Calibration:** It involves a differentiable way of computing quantiles (mainly how to sort them in a differentiable way (Blondel et al., 2020; Cuturi et al., 2019; Williamson, 2020)).

CHAPTER 2. METHODOLOGY: CONFORMAL PREDICTION FOR REGULARIZATION

The loss function used in ConfTr paper is a combination of functions aimed at reducing inefficiency while improving classification abilities:

$$\min_{\omega} \log(\mathbb{E}[\mathcal{L}(C_{\omega}(X; \tau), Y) + \lambda \Omega(C_{\omega}(X; \tau))])$$

with

$$\Omega(C_{\omega}(X; \tau)) := \max \left(0, \sum_{k=1}^K C_{\omega,k}(x, \tau) - \kappa \right),$$

a “smooth” size loss intended to minimize the expected inefficiency, κ is a hyperparameter and $\mathcal{L}$ a generic loss (in this case a classification loss to enforce coverage)

$$\mathcal{L}_{class}(C_{\omega}(x; \tau), y) := \sum_{k=1}^K L_{y,k} \left[\underbrace{(1 - C_{\omega,k}(x, \tau)) \cdot \delta[y = k]}_{\text{enforce } y \text{ to be in } C} + \underbrace{C_{\omega,k}(x, \tau) \cdot \delta[y \neq k]}_{\text{penalize class } k \neq y \text{ not to be in } C} \right]$$

That is the loss used in ConfTr algorithm for constructing confidence sets for classification purposes.

We report some results obtained by ConfTr on several datasets of interest:

Inefficiency ↓, ConfTr (trained w/ THRLP), $\alpha = 0.01$							
	THR				APS		
Dataset	Basel.	Bel	ConfTr	+$\mathcal{L}_{class}$	Basel.	ConfTr	+$\mathcal{L}_{class}$
MNIST	2.23	2.70	2.18	2.11 (-5.4%)	2.50	2.16	2.14 (-14.4%)
F-MNIST	2.05	1.90	1.69	1.67 (-18.5%)	2.36	1.82	1.72 (-27.1%)
EMNIST	2.66	3.48	2.66	2.49 (-6.4%)	4.23	2.86	2.87 (-32.2%)
CIFAR10	2.93	2.93	2.88	2.84 (-3.1%)	3.30	3.05	2.93 (-11.2%)
CIFAR100	10.63	10.91	10.78	10.44 (-1.8%)	16.62	12.99	12.73 (-23.4%)

Figure 2.1.3: Inefficiency results on several datasets with ConfTr [3]

It is clear that including conformal prediction during training helps decreasing the inefficiency while improving coverage. Using a task-specific loss class improves slightly the ability of ConfTr, most likely on datasets that include several classes.

2.2 Regularization for Regression Problems

We focus now on regression problems. In this work, we do not try to create adaptive confidence intervals, we instead try to obtain as narrow as possible confidence intervals for prediction purposes while enforcing coverage guarantees.

The main idea is the same as the one presented during the analysis of ConfTr algorithm, including differentiable prediction and calibration during the training phase of the regression model. The most noticeable differences are the loss function and the non-conformity scores used. We use:

$$S(X; Y) := |Y - \hat{f}(X)| \text{ as non conformity scores}$$

CHAPTER 2. METHODOLOGY: CONFORMAL PREDICTION FOR REGULARIZATION

and

$$L_{1Batch} = \frac{1}{n} \sum_{i=1}^n \max((\hat{y}_i - \hat{\tau}) - y_i, y_i - (\hat{y}_i + \hat{\tau}))$$

$$L_{2Batch} = 2 * \hat{\tau}$$

$$Loss_{custom} = L_{1Batch} + \lambda * L_{2Batch} \text{ as loss function}$$

The non conformity score function is used to measure how non-conformal our prediction is by calculating the absolute value of difference between the prediction and the true value. The loss function we use is composed by L_{1Batch} , which evaluates how badly the confidence interval misses the target value, we sum all over the maximum errors committed on the points in the batch and we then take the average value. L_{2Batch} is the confidence interval's width that we try to minimize while enforcing the coverage guarantee required by the user and λ is a scaling factor to weight the interval's width. We choose to use the custom loss over the classical MSE loss since we can both minimize the prediction error (the more erroneous our prediction is, meaning that the true value falls out of the predicted confidence interval, the more the model has to correct its prediction) and minimize the interval's width.

Feature	Custom Loss (L_{custom})	MSE
Error Metric	Penalizes when true value is outside the interval	Penalizes all deviations from true value
Interval Width Control	Yes, via $L_{2Batch} = 2 \cdot \hat{\tau}$	No
Penalty on Large Errors	Penalizes large interval misses, no error inside intervals	Penalizes large errors quadratically
Regularization Term	Controls interval width (L_{2Batch})	No
Purpose	Suited for conformal prediction	Suited for point prediction

Table 2.2.1: Comparison of Custom Loss vs MSE

CHAPTER 2. METHODOLOGY: CONFORMAL PREDICTION FOR REGULARIZATION

Algorithm 1 Conformal Training for regression

```

1: for mini-batch  $B=(inputs, targets)$  in trainloader do
2:   Randomly split batch:
3:    $B_{cal} + B_{pred} = B$ 
4:    $confscores \leftarrow CONFSCORES(\{\hat{y}_i, y_i\}_{i \in B_{cal}})$ 
5:    $\tau \leftarrow quantile(confscores, 1 - \alpha)$ 
6:   Calculate loss:
7:    $\mathcal{L}_B \leftarrow \sum_{i \in B_{pred}} \mathcal{L}_1(\{\hat{y}_i, y_i\}_{i \in B_{pred}}) + \mathcal{L}_2(\lambda, \tau)$ 
8:    $\Delta \leftarrow \nabla_{\theta}(\mathcal{L}_B)$ 
9:   Update model parameters with  $\Delta$ 
10: end for

```

Algorithm 2 CONFSCORES($\{\hat{y}, y\}$)

```

1:  $s \leftarrow abs(y - \hat{y})$ 
2: return  $s$ 

```

Figure 2.2.4: Pseudo-code for regression conformal training

2.2.1 Results

We show results of the method applied to some custom datasets. We compare the results obtained through two different ways of training:

- **Basic Training:** we train our neural network to model the data, without enforcing coverage or calculating the desired quantile. Once the model is trained the calculate the non-conformity scores and the quantile;
- **Conformal training:** during the training phase of our neural network we calculate the desired quantile while trying to satisfy three main conditions, reduce the prediction error; satisfy a user defined coverage while a confidence interval as narrow as possible.

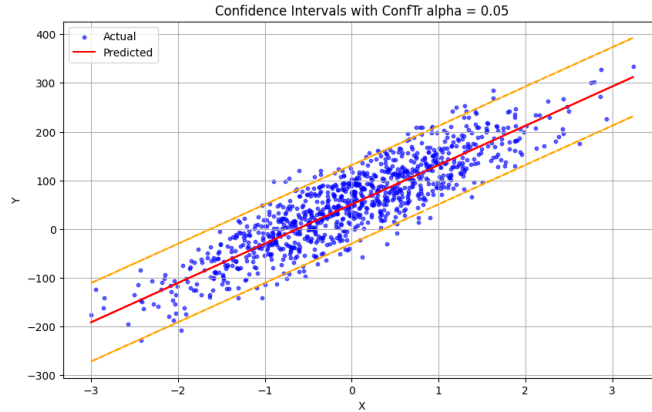


Figure 2.2.5: Conformal training for linear regression

CHAPTER 2. METHODOLOGY: CONFORMAL PREDICTION FOR REGULARIZATION

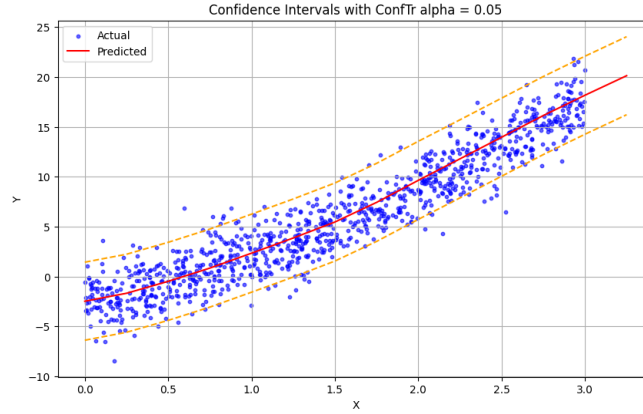


Figure 2.2.6: Conformal training for polynomial regression

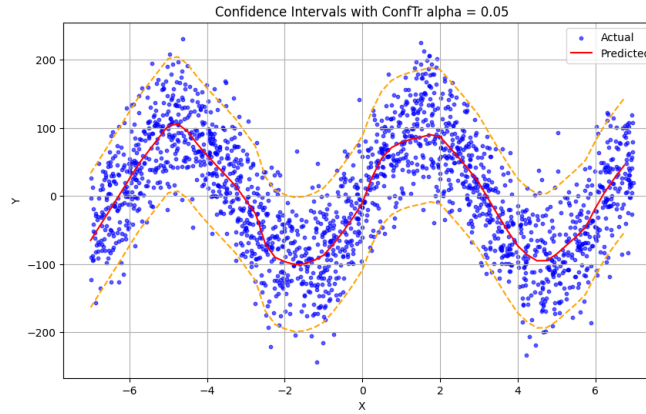


Figure 2.2.7: Conformal training for polynomial regression

Nr	BasicTr		ConfTr	
	τ	Cover	τ	Cover
1 $\alpha = 0.05$	74.6	94.7	74.47	94.68
2 $\alpha = 0.05$	4.08	95.6	3.99	95.25
3 $\alpha = 0.05$	95.88	94.35	97.22	94.8

Table 2.2.2: Comparison between BasicTr and ConfTr

The results that have been discussed in the table above ($\alpha = 0.05$ so a coverage of 95% is expected) show us that ConfTr have a certain effect on the confidence intervals we obtain. In the first example we have, a confidence interval with a width of 149.2 ($2 \cdot 74.6$) and a coverage of 94.7 using the Basic Training algorithm while using the Conformal training version for regression problems we obtain a confidence interval with a width of 148.94 and a coverage of 94.68. In the second example the quantile obtained is slightly different but in the second case (ConfTr) we obtain a coverage closer to the one required. In the third case none of the algorithms achieve the required coverage but the proposed algorithm behaves in the best way.

2.3 Conclusions

In this work we proposed to use conformal prediction (CP) as a regularization term for regression problems. We applied the idea developed by [3] for classification problems, to regression problems with the main intent of providing predictions with a measure of their uncertainty through the use of confidence intervals constructed from a user-requested confidence level. We modified the ConfTr algorithm proposed by updating the non-conformity score function and the loss function in order to be able to obtain the prediction models and user defined confidence intervals for the predictions. The results obtained may suggest that improvements are minimal, but this is mainly due to the fact that the case studies observed were of simple analysis for both models (Basic Training and Conformal Training). It is expected that on more complex problems the improvement may be more significant in obtaining less conservative results. This may be considered as study case for future works.

Bibliography

- [1] Abelardo Montesinos López Osva Antonio Montesinos López and José Crossa. *Multivariate Statistical Machine Learning Methods for Genomic Prediction*. Springer, 2022.
- [2] Anastasios N. Angelopoulos and Stephen Bates. *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification*. 2022. arXiv: 2107.07511 [cs.LG]. URL: <https://arxiv.org/abs/2107.07511>.
- [3] David Stutz et al. *Learning Optimal Conformal Classifiers*. 2022. arXiv: 2110.09192 [cs.LG]. URL: <https://arxiv.org/abs/2110.09192>.
- [4] George C. Runger Douglas C. Montgomery. *Applied Statistics and Probability for Engineers*. Springer, 2022.
- [5] Matteo Fontana, Gianluca Zeni, and Simone Vantini. “Conformal prediction: A unified review of theory and new challenges”. In: *Bernoulli* 29.1 (Feb. 2023). ISSN: 1350-7265. DOI: 10 . 3150 / 21 – bej1447. URL: <http://dx.doi.org/10.3150/21-BEJ1447>.
- [6] Marko Sarstedt and Erik Mooi. *Regression Analysis*. Mar. 2014, pp. 193–233.
- [7] Glenn Shafer and Vladimir Vovk. *A tutorial on conformal prediction*. 2007. arXiv: 0706.3188 [cs.LG]. URL: <https://arxiv.org/abs/0706.3188>.
- [8] Alvin C. Rencher and G. Bruce Schaalje. *Linear Models in statistics*. Springer, 2008.
- [9] Alexander Gammerman Vladimir Vovk and Glenn Shafer. *Algorithmic Learning in a random world*. Springer, 2005.
- [10] Martim Sousa. *Inductive Conformal Prediction: A Straightforward Introduction with Examples in Python*. 2022. arXiv: 2206.11810 [stat.ML]. URL: <https://arxiv.org/abs/2206.11810>.